

AI News Research Tool

Technical Documentation, Architecture & Operations Guide

Submitted By:	Devesh Kumar Mandal
----------------------	---------------------

Submitted To:	Nixthike It solution
----------------------	----------------------

Technologies:	Flask, LangChain, Groq LLM (Llama 3.1), NewsAPI
----------------------	---

Date of Compilation:	June 7, 2026
-----------------------------	--------------

Table of Contents

1. Executive Summary & Overview	3
2. System Architecture & Request Flow	4
3. Core Configuration & Shared Utilities	5
4. Service Layer: Data Fetching & Extraction	6
5. Service Layer: Translation & Exporting	7
6. Web Control & Session State Management	8
7. Project Setup & Local Deployment Guide	9
8. Cloud Deployment & Future Extensions	10

1. Executive Summary & Overview

The **AI News Research Tool** is a production-grade, microservice-inspired Flask web application engineered to simplify news research. In an era of content overload, this tool allows researchers, market analysts, and technology professionals to gather high-quality, real-time news articles on any topic, summarize the developments using high-performance large language models, translate the results into fourteen different languages, and export clean reports for business use.

By integrating **NewsAPI** for real-time article aggregation, **LangChain** for structured pipeline orchestration, and the **Groq Cloud API** (hosting Llama 3.1 8B) for high-speed token generation, the system delivers summaries within seconds. The backend has been written with a strong emphasis on clean code separation, caching policies to minimize API usage costs, and full compatibility with local execution or direct deployment to serverless hosting provider **Vercel**.

2. System Architecture & Request Flow

To ensure scaling and ease of testing, the AI News Research Tool isolates concerns into clear architectural layers. The diagram below represents the sequential flow of data and API interactions triggered when a user initiates a topic search:

```
[ USER BROWSER ]
|
| HTTP POST (Query & Target Language)
v
[ FLASK CONTROLLER (news_research_tool/web.py) ]
|
+---> (1) Fetch raw articles ---> [ NewsAPI SDK (external) ]
|
+---> (2) Generate summary ---> [ LangChain + Groq Cloud (Llama 3.1) ]
|
+---> (3) Translate results ---> [ Groq Cloud (JSON Metadata + Summary Text) ]
|
+---> (4) Exporter Service ---> Writes text & markdown reports to /reports
|
v
[ FLASK RENDERER ] ---> Renders index.html with cached components & download links
```

System Execution Sequence Detail:

- **Aggregation:** The user query is passed to NewsAPI, where we retrieve articles sorted by relevance. The results are immediately sanitized, parsed for content depth, validated to be above a 20-character threshold, and filtered for duplicates based on the title string.
- **Analysis & Summarization:** The sanitized subset of articles is formatted into structured blocks (Title, Description, Source, Content) and sent to Llama 3.1 via LangChain ChatGroq. A custom prompt guides the LLM to extract key developments, trends, and market impacts while preventing hallucinations.
- **Localization (Translation):** If a target language other than English is selected, the application calls the translation service. To optimize speed, titles and descriptions are serialized to JSON and translated in a single batch prompt, preserving structural keys.
- **Persistence (Exporter):** Two files (Markdown and TXT) are generated on-the-fly and stored in the reports folder with unique timestamps, allowing instant downloading via Flask's static routing.

3. Core Configuration & Shared Utilities

The application configures its environment and setup globally to prevent code repetition and guarantee consistency across test environments.

A. Environment Variables & LLM Instantiation (config.py)

Located at `news_research_tool/core/config.py`, this module loads variables from the `.env` file and handles model initialization. It uses `llama-3.1-8b-instant` with a temperature of 0.3 to guarantee deterministic, factual summary generation.

```
import os
from dotenv import load_dotenv
from langchain_groq import ChatGroq
load_dotenv()
GROQ_API_KEY = os.getenv("GROQ_API_KEY")
NEWS_API_KEY = os.getenv("NEWS_API_KEY")
def require_env(value: str, name: str) -> str:
    if value:
        return value
    raise RuntimeError(f"{name} is missing. Add it in environment variables.")
def get_llm() -> ChatGroq:
    return ChatGroq(
        groq_api_key=require_env(GROQ_API_KEY, "GROQ_API_KEY"),
        model_name="llama-3.1-8b-instant",
        temperature=0.3
    )
```

B. Application Logger (logger.py)

The central logging system (`news_research_tool/core/logger.py`) sets up log file appending in `logs/app.log`. It records user queries, application warnings, and API response metrics to assist with system auditing and performance troubleshooting.

C. Shared Paths Resolution (paths.py)

To ensure serverless cross-compatibility, `news_research_tool/core/paths.py` resolves path references dynamically. When deployed on Vercel (indicated by `VERCEL == "1"`), runtime directories are mapped to the writable `/tmp` folder, which is necessary since standard Vercel containers possess a read-only root directory. Otherwise, paths are bound to the project root.

4. Service Layer: Data Fetching & Extraction

Services represent the core business logic. They are decoupled from the Flask HTTP controller to ensure they can be easily unit-tested or integrated into other CLI or GUI clients.

A. Real-Time Article Retrieval (news_fetcher.py)

The news fetcher queries NewsAPI using its python client SDK. It fetches up to ten highly relevant articles matching the user's search query, sanitizes the raw output keys, filters low-quality stubs, and removes duplicate articles published by multiple news outlets. Below is the cleaning and validation code:

```
def clean_article(article: dict) -> dict:
    return {
        "title": article.get("title", "No Title"),
        "description": article.get("description", ""),
        "content": article.get("content", ""),
        "url": article.get("url", ""),
        "source": article.get("source", {}).get("name", "Unknown Source")
    }
def is_valid_article(article: dict) -> bool:
    if not article["description"] or len(article["description"]) < 20:
        return False
    return True
```

B. AI Summarization Pipeline (summarizer.py)

Using LangChain's pipeline expression syntax, summarizer.py pipes a structured layout format containing search topics and formatted articles into the ChatGroq model, parsing the final generation string using StrOutputParser.

```
def generate_summary(query: str, articles: list[dict]) -> str:
    if not articles:
        return "No valid news articles were found."
    formatted_articles = format_articles(articles)
    prompt = create_prompt()
    parser = StrOutputParser()
    # LangChain Pipeline
    chain = prompt | get_llm() | parser
    return chain.invoke({"query": query, "articles": formatted_articles})
```

5. Service Layer: Translation & Exporting

To cater to global audiences and allow immediate offline usage, the service layer supports translation of summaries and formatting exports.

A. LLM-Driven Translation Service (translator.py)

To minimize multiple web latency hops, the translation service converts the long summary briefing and metadata (titles & descriptions) using specialized batch prompts. Metadata is serialized to a JSON array, translated by the LLM in one round-trip, and parsed on the server using robust regex parsing:

```
def translate_articles(articles: list[dict], target_language: str) -> list[dict]:
    target_language = normalize_language(target_language)
    if not articles or not should_translate(target_language):
        return articles
    payload = [{"title": a["title"], "description": a["description"]} for a in articles]
    prompt = PromptTemplate(
        input_variables=["target_language", "articles_json"],
        template="Translate to {target_language}. Return only a JSON array. ... ARTICLES: {articles_json}"
    )
    chain = prompt | get_llm() | StrOutputParser()
    response = chain.invoke({"target_language": target_language, "articles_json": json.dumps(payload)})
    translated_fields = _parse_json_array(response)
    ...
    # Re-maps translated titles and descriptions back to the original dictionary array
```

B. Persisted Report Exports (exporter.py)

The exporter converts generating summaries to formatted TXT and Markdown files, storing them in the directory resolved by the paths system. Document names are automatically sanitized and appended with creation timestamps (e.g., Artificial_Intelligence_20260607_155200.md) to avoid name clashes.

```
def save_as_markdown(query: str, summary: str) -> str:
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    safe_query = clean_filename(query)
    filename = f"{safe_query}_{timestamp}.md"
    filepath = REPORTS_DIR / filename
    with filepath.open("w", encoding="utf-8") as file:
        file.write(f"# AI News Research Report\n\n**Query:** {query}\n\n...")
    return str(filepath)
```

6. Web Control & Session State Management

The user-facing layer (`news_research_tool/web.py`) receives HTTP requests and serves pages. It leverages Flask sessions to track user metrics and memory caching to save costs.

A. Session History Tracking

User query histories and search counts are stored in Flask's client-side cookie session. This is useful for providing immediate feedback on recent activities without requiring database setups. The application saves the last twenty queries and increments search counts, providing a *Clear History* option.

B. Caching Layer to Reduce API Expenses

Since external news searches and large language model translation calls require network hops and incur per-token fees, the controller wraps these service functions using python's built-in `lru_cache`. This guarantees that identical requests do not invoke duplicate API calls within the cache window.

```
@lru_cache(maxsize=32)
def cached_fetch_news(query):
    return fetch_news(query)
@lru_cache(maxsize=32)
def cached_generate_summary(query, articles_json):
    articles = json.loads(articles_json)
    return generate_summary(query, articles)
@app.post("/")
def search():
    query = request.form.get("query", "").strip()
    # ... Retrieves cached news & cached summaries ...
    # ... Performs localization translations and saves reports ...
    # ... Renders template HTML payload ...
```

C. Secure Download Routing

To protect files on the server, the download route uses an absolute boundary check. It verifies that the requested path starts with the resolved reports directory, preventing directory traversal attacks.

7. Project Setup & Local Deployment Guide

This section details setting up a developer environment locally to test and run the AI News Research Tool.

A. Environment Configurations (.env)

Create a file named `.env` in the project root containing the following variables:

```
GROQ_API_KEY=gsk_your_groq_api_key_here
NEWS_API_KEY=your_news_api_key_here
FLASK_SECRET_KEY=generate_a_random_hex_string_for_production
```

B. Local Installation Instructions

Execute these shell commands in your terminal to set up a clean Python virtual environment and install all necessary dependencies:

```
# 1. Create virtual environment
python3 -m venv .venv
# 2. Activate virtual environment (MacOS/Linux)
source .venv/bin/activate
# 3. Install required packages
pip install -r requirements.txt
# 4. Launch the Flask application
python app.py
```

C. Verifying Local Service

Once launched, the terminal displays the active web port. Open `http://127.0.0.1:5001` in your browser. Run a search query such as *Artificial Intelligence in Healthcare*, translate it to *French*, and verify that the summaries render and can be downloaded as MD/TXT.

8. Cloud Deployment & Future Extensions

The codebase has been designed for serverless operations, allowing zero-maintenance deployments.

A. Serverless Deployments via Vercel

The project includes configuration files for hosting on the Vercel cloud serverless platform. The `vercel.json` routing block directs all HTTP traffic to the endpoint script `api/index.py`, which wraps the Flask web object:

```
{
  "version": 2,
  "builds": [
    { "src": "api/index.py", "use": "@vercel/python" }
  ],
  "routes": [
    { "src": "/*", "dest": "api/index.py" }
  ]
}
```

Vercel Configuration Steps:

1. Install Vercel CLI: `npm install -g vercel`.
2. Run `vercel login` to authenticate.
3. Execute `vercel` from the root folder, linking to your project.
4. Navigate to Vercel dashboard settings, add your Environment Variables (`GROQ_API_KEY`, `NEWS_API_KEY`, `FLASK_SECRET_KEY`), and deploy.

B. Future Extensions Roadmap

- **Unified PDF Exports:** Add PDF generation directly into the web UI using this layout setup, allowing users to download summaries as premium PDF reports.
- **Persistent Query Storage:** Migrate from client-side cookies to a database (e.g. Supabase, SQLite, PostgreSQL) to allow querying across sessions and devices.
- **Interactive Data Charts:** Integrate Python's Matplotlib or browser-based Chart.js to map sentiment trends or source distribution visually for research topics.
- **Scheduling & Alerts:** Enable cron scheduling to run automated summaries for specific queries (e.g., daily market update) and send notifications or emails.